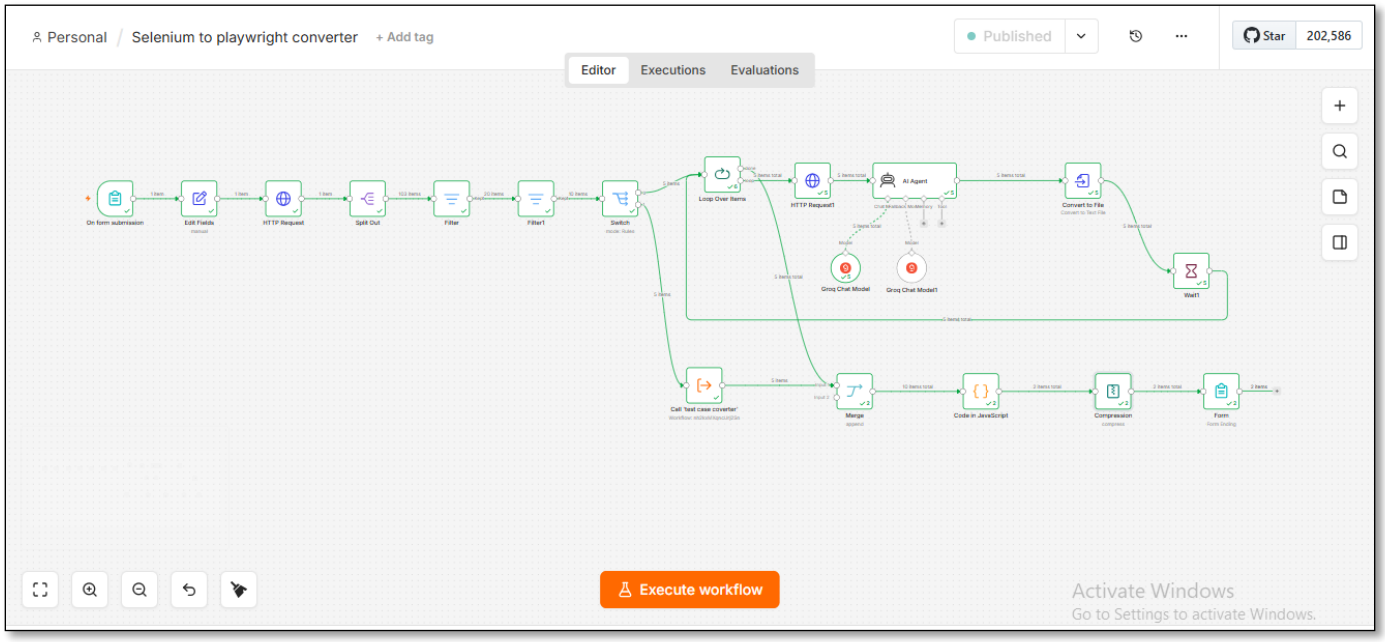
Selenium to Playwright Converter – n8n Workflow

Issues Identified, Resolutions and Key Takeaways

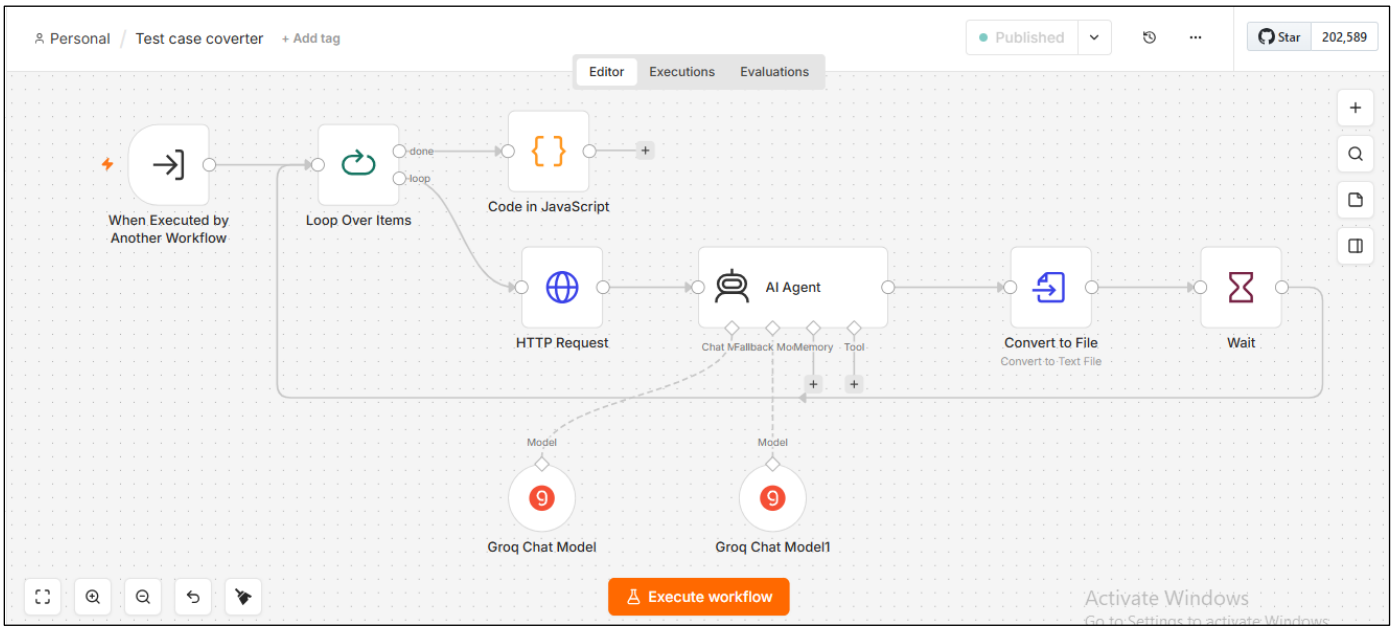
Purpose. This document records the key issues encountered while building and testing the Selenium-to-Playwright conversion workflow in n8n, the resolution applied for each issue, and the practical lessons learned for future workflow development and troubleshooting.

1 Framework

1.1 Main workflow



1.2 Sub workflow



1.3 Output file after migration

.txt

2 Issues Encountered and Resolutions

2.1 HTTP Request1 was not executed

- **Issue:** n8n displayed “Node was not executed” because the execution path did not reach HTTP Request1.
- **Root Cause / Analysis:** The workflow path was traced backward from HTTP Request1. The Loop Over Items node was found to be connected through the wrong output.
- **Resolution:** HTTP Request1 was connected to the loop output of Loop Over Items so that each incoming item could be processed.
- **Key Takeaway:** A node showing “not executed” is not necessarily faulty; first verify that the execution path actually reaches the node.

2.2 Loop Over Items had no loop output connection

- **Issue:** The Loop Over Items node indicated that no node was connected to its loop output.
- **Root Cause / Analysis:** The downstream HTTP Request1 node was not connected to the loop branch.
- **Resolution:** The connection was changed to Loop Over Items → loop → HTTP Request1.
- **Key Takeaway:** Understand the purpose of the loop and done outputs. The loop output processes items; the done output is used after the loop completes.

2.3 Switch rule did not route page files

- **Issue:** The Switch rule used “is equal to /pages”, while the actual GitHub path was a complete file path containing /pages, such as a Java file path under the pages directory.
- **Root Cause / Analysis:** The condition was comparing the entire path with the short directory value.
- **Resolution:** The routing condition was changed from “is equal to” to “contains”, with /pages retained as the matching value.
- **Key Takeaway:** For hierarchical file paths, “contains” can be more appropriate than exact equality when the target directory appears within a longer path.

2.4 Test-case branch was not routing correctly

- **Issue:** The test-case routing rule also used exact equality for /testcases, so full test-case file paths were not matched.
- **Root Cause / Analysis:** The same path-matching issue existed in the second Switch rule.
- **Resolution:** The rule was changed from “is equal to” to “contains”, using /testcases as the matching value.
- **Key Takeaway:** Use routing conditions that reflect the actual structure of incoming data rather than assuming the field contains only the directory name.

2.5 Sub-workflow item-processing mode needed adjustment

- **Issue:** The test-case branch needed to process incoming test-case files individually.
- **Root Cause / Analysis:** The Execute Sub-workflow node was configured to process one incoming item at a time.
- **Resolution:** The execution mode was changed to “Run once for each item”.
- **Key Takeaway:** Choose between processing all items together and processing each item individually based on the data-processing requirement.

2.6 Convert to File reported that Output was not set

- **Issue:** The Convert to File node expected a field named Output, while the AI Agent returned the field as output.
- **Root Cause / Analysis:** The field-name mismatch was identified.
- **Resolution:** The Text Input Field was changed from Output to output.
- **Key Takeaway:** n8n field names are case-sensitive. Always verify the exact field name and structure produced by the previous node.

2.7 JavaScript Code node reported an unexpected '&' token

- **Issue:** The JavaScript contained HTML-encoded/escaped characters such as   and escaped brackets, which are invalid in the intended JavaScript syntax.
- **Root Cause / Analysis:** The actual code content was inspected rather than changing the workflow logic.
- **Resolution:** The corrupted characters were replaced with clean JavaScript syntax while preserving the intended logic for merging binary inputs.
- **Key Takeaway:** When a syntax error appears unexpectedly, inspect the literal characters in the code. Copy/paste or formatting transformations can introduce invalid characters.

2.8 Compression node could not identify the binary fields

- **Issue:** The expression used to obtain the binary field names was being interpreted as text instead of as an n8n expression.
- **Root Cause / Analysis:** The Input Binary Field(s) configuration was checked.
- **Resolution:** Expression mode was enabled so that {{ Object.keys($binary).join(',') }} was evaluated dynamically.
- **Key Takeaway:** In n8n, distinguish clearly between literal values and expressions. Expressions must be entered/evaluated in Expression mode.

2.9 ZIP contained an AI request for source code instead of converted content

- **Issue:** The Test Case Converter AI reported that it needed the Java test-case source code. The HTTP Request node responsible for retrieving the source file was deactivated.
- **Root Cause / Analysis:** The sub-workflow execution and node status were inspected.
- **Resolution:** The deactivated HTTP Request node was enabled, allowing the Java source content to reach the conversion flow.
- **Key Takeaway:** If an AI node says it is missing input, check upstream data-retrieval nodes and confirm that they are active and actually executed.

3 Overall Debugging Takeaways

3.1 Trace the data path:

When a node fails or is skipped, first determine whether the expected input actually reached that node.

3.2 Validate each node independently:

Execute upstream nodes and inspect their output before changing downstream configuration.

3.3 Check exact field names:

Confirm spelling, capitalization, nesting and data type of fields referenced in expressions.

3.4 Understand n8n routing:

Switch, Filter and Loop Over Items determine which path receives the data. Incorrect routing can make downstream nodes appear broken.

3.5 Use sub-workflows for modularity:

Separate page conversion, test-case conversion and other responsibilities when the solution becomes complex.

3.6 Verify node status:

A deactivated node can silently prevent required data from reaching the next stage.

3.7 Test incrementally:

Make one change, execute the affected section, confirm the result, and then proceed to the next step.

4 Final Workflow Learning

The main lesson from this implementation is: debug the workflow by following the data from node to node. For each stage, confirm three things: **Is the node being reached? Is the expected data entering it? Is it producing the expected output?** Once these checks are performed systematically, most n8n workflow issues can be isolated without changing multiple nodes at the same time.